vulpes



PLANO · DECISÕES TOMADAS A 14/11/2026

Vulpes Flos 1.0

O Vulpes 0.1, quase três vezes maior e quatro vezes mais lido, treinado só com as GPUs gratuitas do Kaggle.

14 de novembro de 2026 · até esta data chamava-se Aupus 1

Sucede ao Vulpes 0.1 (125M, treinado a 10/11/2026); vem antes do Vulpes Lusitanus 1.0 (plano à parte)

Regra acima de todas: tirar o máximo do Kaggle grátis. Tempos e resultados futuros são estimativas, a confirmar no K5.

EM RESUMO

O melhor Vulpes que se pode treinar sem pagar

O Vulpes Flos 1.0 é o 0.1 levado até onde as GPUs gratuitas do Kaggle deixam: **um modelo maior, que lê mais texto e se lembra de mais conversa.** Continua a ser feito neste repositório, em PyTorch simples, com os mesmos dados escritos por pessoas, o mesmo tokenizer e o mesmo chat.

348 M	2 048	10–15 B+	0 USD
parâmetros 2,8× o Vulpes 0.1; cabe à vontade na tua GPU de 4 GB	**tokens de contexto** 2× o 0.1; conversas mais longas no chat	**tokens de treino** 4–6× o 0.1; continua enquanto a perda melhorar	**custo** duas T4 do Kaggle e, se o ensaio correr bem, as TPUs

Três peças

O modelo

Vulpes Flos 1.0 Base e **Vulpes Flos 1.0 Instruct**: o preset `base` (348M) que já existe e está testado desde o marco 4, agora com contexto de 2 048.

Os dados

O **sample-10BT inteiro** do FineWeb-Edu: ~9,5B tokens, mais de 3× os 2,94B já construídos, com a mesma pipeline, os mesmos filtros e o mesmo tokenizer.

O sítio

Kaggle, duas T4, em sessões de até 12 h encadeadas, cada uma a retomar a anterior. A **quota de TPU**, separada, é ensaiada no K5 (página 5).

Como se chega lá

Em quatro fases e dez marcos (página 11), todos gratuitos. A regra do 0.1 continua: **nada longo sem um ensaio curto primeiro**. Antes das semanas de treino, um benchmark de uma hora mede a velocidade real das T4 e um piloto de duas sessões prova que a retoma entre sessões funciona.

> **Expectativa honesta.** 348M continua a ser pequeno. O Vulpes Flos 1.0 deve escrever inglês mais coerente e acertar mais perguntas simples do que o 0.1, mas continua a inventar factos, não programa e quase não fala português. O ganho esperado é real e modesto (~5–10 pontos no HellaSwag), e demora dois a três meses de quota de GPU, menos se as TPUs renderem.

O que ficou decidido

A 14/11/2026 ficaram decididas seis das sete decisões (página 13), todas como recomendado e com uma regra por cima: **tirar o máximo do Kaggle**. Por isso o treino não para nos 10B, nenhuma hora de quota de GPU vai para o que o PC pode fazer, e as TPUs são ensaiadas. Só a publicação dos pesos fica para o fim. O que o Vulpes Flos 1.0 constrói (DDP, WSD, fp16, o fluxo do Kaggle) passa depois para o Vulpes Lusitanus 1.0.

1 · PONTO DE PARTIDA

O que o Vulpes 0.1 deixa por melhorar

O 0.1 provou a cadeia inteira, do tokenizer ao chat. Os seus limites dividem-se em dois grupos: os que se resolvem com mais computação, que o Kaggle dá de graça, e os que pedem um modelo diferente.

Resultados medidos

	VULPES 0.1 BASE	VULPES 0.1 INSTRUCT	AO ACASO
HellaSwag (acc_norm, 10 042 perguntas)	31,3%	31,2%	25%
ARC-Easy (acc, 2 376)	52,4%	52,2%	25%
PIQA (acc_norm, 1 838)	61,9%	61,0%	50%
Perplexidade de validação (FineWeb-Edu)	22,9	24,8	32 000

Fonte: `runs/vulpes-0.1-*/eval_best.json`. Base: 2,5B tokens, 9 537 passos, ~4 h numa A100 80 GB a ~167 mil tokens/s (138 TFLOPS efetivos). Instruct: 600 passos de SFT.

O que se resolve com mais computação: o Vulpes Flos 1.0

1. É pequeno

125M é o tamanho do GPT-2 pequeno. A este nível, crescer é o ganho mais barato por FLOP, e 348M ainda corre em fp32 no teu GTX 1650 Super.

2. Leu pouco

2,5B tokens, 0,85 épocas dos 2,94B construídos. O sample-10BT do FineWeb-Edu tem mais do triplo, e o 0.1 só usou 4 dos 14 ficheiros.

3. Esquece depressa

1 024 tokens de contexto: no chat, a conversa enche a memória ao fim de poucas trocas. 2 048 custa só +8% de FLOP por token.

4. O fim do treino era fixo

Com o cosseno, o fim tinha de ser escolhido à partida. Com quota semanal e sessões de 12 h, convém poder decidir a meio quando parar.

O que fica para o Vulpes Lusitanus 1.0

- **Código e português**: precisam de um tokenizer novo (hoje 1,85 e 2,8 caracteres por token) e de dados novos.
- **Ferramentas e o agente**: precisam de tokens especiais novos e de um modelo de ~1B.
- **1B parâmetros**: não cabe numa T4 nem na quota de GPU do Kaggle (página 4).

2 · O SÍTIO

O que cabe no Kaggle grátis

O Kaggle dá duas GPUs T4 algumas horas por semana, sem cartão. Somadas ao longo de dois meses, dão cerca de 15 vezes a computação do treino do 0.1. Esta página conta só com as T4, o caminho garantido; as TPUs vêm na página seguinte.

O que o Kaggle dá

O QUÊ	VALOR	O QUE SIGNIFICA PARA O VULPES
GPU	2× T4 de 16 GB (ou uma P100)	fp16 com tensor cores, sem bf16: o treino usa fp16 com GradScaler, que o código já tem (decisão 13 do 0.1)
Quota	~30 h por semana	O valor exato não vem na documentação e vê-se na conta; há relatos de 30–41 h. As contas abaixo usam 30 h
Sessão	Até 12 h, em segundo plano	"Save & Run All" corre sem o browser aberto; `--stop-after` para antes do limite
Disco guardado	20 GB em `/kaggle/working`	Dois checkpoints de 4,2 GB e o `best.pt` cabem
Dados	Datasets privados, só de leitura	Os shards (~19 GB) entram como dataset, montado em `/kaggle/input`

A conta

OPÇÃO	FLOP	HORAS EM 2×T4	SEMANAS	VEREDICTO
Refazer o 0.1 (125M, 2,5B)	2,1e18	17–23	< 1	Já existe
Continuar o 0.1 até 10B tokens	6,2e18	50–70	~2	Barato, ganho pequeno; plano B
Vulpes Flos 1.0: 348M, 10B, contexto 2k	2,4e19	195–270	6,5–9	**Recomendado**
Vulpes Flos 1.0 continuado a 15B	3,7e19	290–410	10–14	**Decidido (D2)**, se a perda ainda descer
Lusitanus Mini (321M, 10B, 4k)	2,5e19	200–280	7–9	Precisa de tokenizer novo
Vulpes Lusitanus 1.0 (1B, 50B)	3,8e20	3 000–4 200	100–140	Não cabe numa T4

Com 25–35 TFLOPS efetivos para as duas T4 em fp16 com DDP: uma **estimativa, não uma medição** (~¼ dos 138 TFLOPS medidos na A100). O K5 mede o valor real e refaz esta tabela. FLOP com `vulpes.model.config.flops_per_token`.

Porque não o 1B aqui. Com AdamW em fp32, cada parâmetro ocupa 16 bytes (pesos, gradientes, dois momentos): 1B são ~17 GB por GPU, e uma T4 tem 16 GB. O DDP copia o modelo para as duas GPUs, por isso não soma memória. E mesmo com sharding, seriam dois anos de quota.

3 · A OUTRA QUOTA

As TPUs do Kaggle: um ensaio que pode valer muito

Além da quota de GPU, o Kaggle tem uma quota de TPU separada. Se o PyTorch do Vulpes correr bem nelas, cada semana pode dar várias vezes mais treino. Se não correr, não se perde nada: o plano assenta nas T4.

As duas quotas, lado a lado

	GPU (O PLANO)	TPU (O ENSAIO)
Hardware	2× T4, 16 GB cada	TPU v5e-8 (ou v6e-8, segundo uma fonte de 2026): 8 chips de 16 GB
Quota	~30 h por semana	~20 h por semana, à parte
Sessão	até 12 h	~9 h
Precisão	fp16 com GradScaler	bf16, como na A100 do 0.1
Como corre o PyTorch	CUDA, como hoje	PyTorch/XLA: o mesmo modelo, compilado para a TPU
Pico	~130 TFLOPS fp16	~1 600 TFLOPS bf16 (v5e-8)
Efetivo (estimativa)	25–35 TFLOPS	200–300 TFLOPS, se só 13–19% do pico
Tokens do Vulpes Flos 1.0 por semana	1,1–1,5 B	**6–9 B**

Os valores das TPUs vêm de fontes secundárias que não concordam entre si (modelo, horas, duração da sessão). Confirmam-se no painel da conta antes do K5. O rendimento real do PyTorch/XLA com o nosso código é a grande incógnita, e é isso que o ensaio mede.

O ensaio, dentro do K5

1. Correr o Vulpes Flos 1.0 com PyTorch/XLA numa TPU, em bf16. O pré-treino tem formas fixas, que é o que o XLA quer.
2. Medir tokens/s com 1 e com 8 chips.
3. Treinar 200 passos e comparar a perda com os mesmos 200 passos nas T4.
4. Guardar na TPU e retomar na T4, e o contrário.

Gasta só quota de TPU, que de outra forma ficava parada, por isso não atrasa as T4.

Não muda as regras do projeto: continua a ser PyTorch, sem `transformers` nem `accelerate`. O código XLA fica num caminho à parte e o caminho CUDA não muda.

O que acontece depois

Se correr bem

O mesmo run alterna: primeiro a quota de TPU, a mais rápida, depois a de GPU, nunca as duas ao mesmo tempo. A essa velocidade, o limite deixa de ser a computação e passa a ser os dados (~9,5B tokens únicos). Antes do treino longo, o K5 revê o tamanho (um modelo maior, perto de ~700M) e os dados (mais ficheiros do FineWeb-Edu), com os números medidos à frente.

Se correr mal

Fica tudo nas T4, exatamente como nas páginas 4, 8 e 9. Perdem-se algumas horas de quota de TPU e um dia de trabalho, e o calendário do plano não muda.

4 · ARQUITETURA

O mesmo desenho, um preset acima

A família Llama do 0.1 não muda. O Vulpes Flos 1.0 é o preset `base`, que existe desde o início do projeto e cuja contagem de parâmetros e FLOP está testada contra o modelo real.

	VULPES 0.1	VULPES FLOS 1.0
Parâmetros (sem embeddings)	125,3M (100,7M)	348,4M (315,7M)
Camadas × largura	16 × 768	28 × 1 024
Cabeças de query / de KV (GQA)	12 / 4	16 / 4
Dimensão por cabeça	64	64
SwiGLU (camada escondida)	2 048	2 816
Vocabulário	32 000	32 000 (o mesmo tokenizer)
Contexto	1 024	2 048
RoPE θ	10 000	10 000
FLOP por token de treino	0,83 G	2,44 G
Tokens de pré-treino	2,5 B	10–15 B+, enquanto a perda descer
Tokens por parâmetro	20	29–43+

Parâmetros e FLOP calculados com `vulpes.model.config.param_count` e `flops_per_token`, a mesma contabilidade do 0.1.

Porquê esta forma

348M, não 125M nem 1B

Para a computação disponível (~$2{,}4\times10^{19}$ FLOP em dois meses), a regra do Chinchilla aponta para ~450M com 9B tokens. Um pouco mais pequeno e mais treinado custa quase nada em qualidade e corre mais depressa no teu PC. Se as TPUs renderem, o K5 revê o tamanho.

Fundo e estreito

28 camadas de 1 024, como o MobileLLM-350M: a este tamanho, mais camadas rendem mais do que mais largura. É a mesma lógica que deu 16 × 768 ao 0.1.

O mesmo tokenizer

Os 2,94B tokens já construídos servem tal como estão, e o chat, a avaliação, o SFT e a exportação não mudam. Trocar o tokenizer é o primeiro passo do Lusitanus, não deste.

2 048 tokens, θ = 10 000

θ = 10 000 chega bem a 2k (o Llama 2 usou-o a 4k). O dobro do contexto custa +8% de FLOP por token e dá conversas duas vezes mais longas no chat.

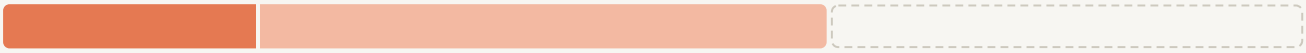
No teu PC

Pesos em fp32: 1,4 GB; cache de 2 048 tokens: ~117 MB. Cabe com margem nos ~2,9 GB livres do GTX 1650 Super, sem fp16 (que é lento nesta GPU, decisão 13). Estimativa a medir: 30–60 tokens/s.

5 · DADOS

O FineWeb-Edu inteiro, não um quarto

O 0.1 usou os primeiros 4 dos 14 ficheiros do sample-10BT. O Vulpes Flos 1.0 usa os 14, com a mesma revisão fixa, a mesma pipeline e as mesmas regras: texto escrito por pessoas, com licença aberta.



2,94 B já construídos **~6,6 B** novos · total ~9,5 B **15 B+** segunda passagem

Já construídos (ficheiros 1–4) Novos (5–14) Segunda passagem, enquanto a perda descer (D2)

Tokens de treino. Os ~9,5 B são estimados a partir dos 2,94 B dos primeiros 4 ficheiros; os duplicados crescem com a escala (§16 do PLAN), por isso o valor real sai um pouco abaixo.

FONTE	O QUE É	LICENÇA	TOKENS
FineWeb-Edu sample-10BT, 14 ficheiros	Páginas educativas em inglês, como no 0.1. Os 4 primeiros ficheiros dão os mesmos documentos e o mesmo conjunto de validação de antes (a divisão é por hash), por isso os números ficam comparáveis.	ODC-By 1.0	~9,5 B

Como se constrói

No teu PC, como no M7

`download_fineweb_edu.py --files 14` (28,5 GB, ~1,5 h) e depois `prepare_data.py` com 3 processos e prioridade abaixo do normal (decisão 38): ~4 h. Saem ~19 GB de shards; os 8,6 GB de parquet do 0.1 podem ser apagados.

Nenhum código novo

Filtros Gopher, deduplicação exata e MinHash, divisão por hash, shards `uint16`: tudo como no 0.1. A deduplicação a 9,5B tokens pede ~5 GB de RAM, dentro dos 16 GB do desktop.

Para o Kaggle

Os shards sobem como um dataset privado (D4, decidido), lido pelas sessões de GPU e de TPU. Se o upload for demasiado lento, a mesma pipeline corre num notebook de CPU do Kaggle, que não gasta quota de GPU.

Repetir dados

Até ~9,5B tokens nenhuma janela se repete; 15B são ~1,6 épocas e 19B duas. Muennighoff et al. (2023) mostram que até ~4 épocas quase não se perde face a dados novos, por isso continuar além dos 10B vale a pena.

6 · PRÉ-TREINO

Primeiro medir, depois treinar

Três etapas, todas no Kaggle: uma hora para medir, duas sessões curtas para provar a retoma, e depois o treino longo, sessão a sessão.

ETAPA	O QUÊ	QUOTA	PARA QUÊ
K5 · Benchmark	348M em 1 e 2 T4; ensaio na TPU	~1 h GPU ~3 h TPU	tokens/s reais, micro-batch, checkpointing, `torch.compile`; o ensaio da página 5; refaz o calendário
K6 · Piloto	348M, 2 sessões de ~2 h	~4 h	A segunda sessão retoma a primeira e repete a perda dos mesmos passos, como no piloto do 0.1
K7 · Base	348M, 10–15B+ tokens	195–410 h	O treino principal: 2–3 sessões por semana, enquanto a perda descer

Lote de 256 sequências × 2 048 = 524 288 tokens por passo (o do GPT-3 Medium, 350M): ~19 000 passos para 10B, ~28 600 para 15B.

Hiperparâmetros

AdamW como no 0.1 (β 0,9/0,95, decay 0,1 só nas matrizes, clip 1,0). **LR máxima 3×10^{-4}**, a do GPT-3 Medium (o 0.1 usou 6×10^{-4}: modelos maiores querem LR menor), aquecimento de 500 passos.

fp16 com GradScaler nas T4, que não têm bf16 (nas TPUs, bf16). O código já está preparado: o RMSNorm e a perda calculam-se em fp32, e uma perda não finita para o treino com uma mensagem clara.

Agenda WSD (aquecimento, patamar, descida) em vez de cosseno. No patamar, a LR não depende de `max_steps`, por isso a meta (10B ou 15B) pode mudar a meio sem recomeçar. A descida ocupa os últimos ~15% e só começa quando a perda deixar de compensar a quota (D2). Continua a ser uma função pura do passo (decisão 21).

Duas GPUs com DDP (`torchrun`), em PyTorch simples. Cada passo vê as mesmas sequências, pela mesma ordem, que numa só GPU, por isso o treino pode passar para uma GPU, para o PC ou para um pod sem mudar os dados.

Calendário

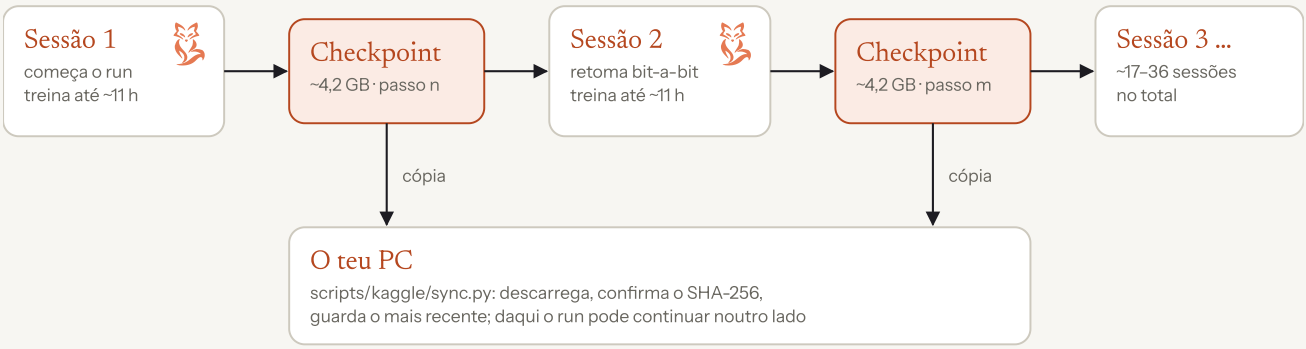
FASE	QUOTA	SEMANAS A 30 H
K4–K6 · Preparar o Kaggle, benchmark, piloto	~6 h	< 1
K7 · Base, 10B tokens	195–270 h	6,5–9
K7 · continuação até 15B (D2)	+95–140 h	+3–5
K8–K9 · Avaliação (no PC) e Instruct	~1–2 h	< 1

Só com as T4. Se as TPUs renderem o estimado, os 15B cabem em ~2–3 semanas de quota de TPU. Para comparar: o K7 a 10B numa A100 alugada seriam ~50 h (~60–95 USD).

7 · O CICLO DE SESSÕES

Doze horas de cada vez

Nenhuma sessão do Kaggle chega para o treino, e nenhuma precisa de chegar. O 0.1 já retomava bit-a-bit noutra máquina; aqui, cada sessão é uma retoma.



Cada sessão retoma o checkpoint mais recente, treina até ~11 h 15 min, guarda e termina. O checkpoint passa para a sessão seguinte e para o teu PC, verificado com SHA-256. Os dados ficam num dataset à parte e são lidos por todas.

Uma sessão, passo a passo

1. Clona o repositório na versão fixada e instala o pacote (o PyTorch do Kaggle já vem instalado).
2. Procura o checkpoint mais recente nos dados de entrada; se não houver nenhum, é o início do run.
3. `torchrun --nproc_per_node 2 scripts/train.py --resume … --stop-after 11.25`, que guarda a cada ~30 min.
4. No fim, ou ao receber o aviso de paragem, guarda o checkpoint em `/kaggle/working` e termina.
5. No PC, um `scripts/kaggle/sync.py` (como o do RunPod) descarrega-o pela API do Kaggle e confirma o SHA-256.

O que o código ganha

No treino (K2)

- **DDP**: o carregador parte cada passo entre as GPUs; a soma de gradientes só se faz no último micro-batch; só a GPU 0 escreve logs e checkpoints.
- **WSD** em `optim.py`, ao lado do cosseno, que fica para o 0.1.
- Configuração `configs/train/flos-kaggle.yaml` e testes em CPU, como sempre (DDP com 2 processos em CPU).

No Kaggle (K4–K5)

- **`scripts/kaggle/`**: o notebook, o arranque da sessão e o `sync.py` do PC.
- Como o checkpoint passa de uma sessão para a seguinte (output do notebook ou dataset privado pela API) escolhe-se no K4, medindo o tempo de cada um.
- Um caminho PyTorch/XLA para as TPUs (K5), à parte do caminho CUDA.
- Nunca há duas sessões a escrever no mesmo run.

Nenhuma hora de quota desperdiçada

- **A quota de GPU é só para treinar** (pré-treino e SFT). Dados, avaliação e cópias fazem-se no teu PC ou num notebook de CPU, que não gasta quota.
- **Toda a quota, todas as semanas:** 2–3 sessões seguidas até a quota acabar; a quota de TPU usa-se primeiro, se o ensaio correr bem.
- **A velocidade máxima, medida:** o K5 escolhe o micro-batch, o `torch.compile` e o checkpointing mais rápidos que cabem na memória, antes do treino longo.

8 · INSTRUCT E AVALIAÇÃO

Conversar melhor, medido da mesma forma

O Instruct repete o SFT do 0.1 com um modelo maior e conversas duas vezes mais longas. A avaliação usa os mesmos testes e o mesmo conjunto de validação, para que a comparação com o 0.1 seja direta.

Fine-tuning (SFT)

DADOS	O QUE ENSINAM	LICENÇA
oasst2, inglês (5 270 conversas)	Conversar; o melhor ramo de cada árvore, como no 0.1	Apache-2.0
Dolly 15k (15 011)	Instruções curtas, como no 0.1 (decisão 31)	CC BY-SA 3.0
Identidade (44, repetida 3×)	Quem é o Vulpes; atualizada por ti para dizer que é o Vulpes Flos 1.0	Apache-2.0

LR 3×10^{-5}, perda só nas respostas, paragem antecipada pela perda de validação. Com 2 048 tokens, quase nenhuma conversa é cortada. Numa T4: ~1–2 h. A avaliação corre no teu PC (o modelo cabe em fp32), sem gastar quota.

O que esperar

TESTE	MEDE	0.1 (MEDIDO)	VULPES FLOS 1.0 (ESPERADO)
HellaSwag (acc_norm)	Senso comum	31,3%	36–41%
ARC-Easy (acc)	Ciência escolar	52,4%	56–62%
PIQA (acc_norm)	Física do dia a dia	61,9%	65–68%
Perplexidade de validação	O mesmo conjunto de validação do 0.1	22,9	15–18

As expectativas vêm de modelos publicados de tamanho parecido (GPT-2 Medium, Pythia-410M) e das comparações do FineWeb-Edu, que sobe sobretudo o ARC. Não são promessas: o Pythia-410M viu 30× mais tokens. Com 15B tokens ou mais, contar com a metade de cima dos intervalos.

O que vai conseguir, e o que não

Deve conseguir

- Inglês mais coerente durante mais frases
- Responder bem a mais perguntas simples de escola
- Seguir uma conversa durante mais trocas
- Saber quem é, pelo conjunto de identidade

Não deve conseguir

- Acertar factos com fiabilidade: vai inventar
- Escrever código que funcione
- Conversar bem em português
- Raciocinar em vários passos

9 · MARCOS

Dez marcos, quatro fases

Como no 0.1: um marco de cada vez, cada um com testes e resultados medidos no PLAN. Os marcos chamam-se K1–K10 (de Kaggle), para não se confundirem com os do 0.1 nem com os do Lusitanus.

#	MARCO	ONDE	QUOTA GPU
Fase A · Preparar, no teu PC			
K1	**Plano e decisões** Este documento (decisões tomadas a 14/11); falta o `PLAN.md` do Vulpes Flos 1.0	Desktop	0
K2	**Treino v1.1** DDP, WSD, config do Vulpes Flos 1.0; testes em CPU como sempre	Desktop	0
K3	**Dados** sample-10BT inteiro, ~9,5B tokens, mesma pipeline; dataset privado no Kaggle	Desktop (CPU), ~6 h	0
K4	**Kaggle** Notebook, `scripts/kaggle/`, passagem do checkpoint, cópia verificada para o PC	Desktop + Kaggle	~1 h
Fase B · Ensaio			
K5	**Benchmark e ensaio da TPU** tokens/s reais em 1 e 2 T4; PyTorch/XLA na TPU (página 5); refaz o calendário e, se for o caso, o tamanho	Kaggle	~1 h + ~3 h TPU
K6	**Piloto de duas sessões** A retoma entre sessões repete a perda. Se falhar, para aqui	Kaggle	~4 h
Fase C · O modelo			
K7	**Vulpes Flos 1.0 Base** 348M, contexto 2 048; 10B, depois até 15B ou mais enquanto a perda descer (WSD)	Kaggle, 6,5–14 semanas	195–410 h
K8	**Avaliação** HellaSwag, ARC-Easy, PIQA e perplexidade, contra o 0.1	Desktop	0
Fase D · O assistente			
K9	**Vulpes Flos 1.0 Instruct (SFT)** oasst2, Dolly e identidade; contexto 2 048	Kaggle	~1–2 h
K10	**Release e chat** Exportação, chat no PC, MODEL_CARD, README e PLAN atualizados	Desktop	0

K2 e K3 podem correr ao mesmo tempo: um é código, o outro é o CPU do PC a construir dados. O treino longo (K7) só começa depois do K6.

10 · RISCOS

O que pode correr mal

O 0.1 correu bem porque cada risco tinha uma resposta antes de começar. O mesmo aqui.

RISCO	RESPOSTA
As T4 são mais lentas do que o estimado	O K5 mede antes de começar. A meta passa a ser o que couber: com WSD, parar mais cedo dá um modelo acabado, só menos treinado.
A quota baixa ou o Kaggle muda as regras	O checkpoint está sempre no teu PC. O run continua sem recomeçar noutra máquina (uma GPU alugada ou o desktop), como o 0.1 podia passar do pod para o PC.
Uma sessão acaba a meio	`--stop-after` para antes das 12 h; checkpoints a cada ~30 min. Um erro ou um corte custam no máximo esse tempo.
O fp16 fica instável	GradScaler; RMSNorm e perda em fp32 (já hoje); uma perda não finita para o treino. Retoma-se do checkpoint anterior com LR mais baixa. O piloto mostra-o cedo.
Duas GPUs não escalam bem	O K5 mede 1 contra 2. A soma de gradientes só se faz uma vez por passo. No pior caso, uma GPU: metade da velocidade, o dobro das semanas.
Sai uma P100 em vez das T4	Escolher sempre “GPU T4 ×2”. A P100 não tem tensor cores e as versões recentes do PyTorch podem já não a suportar.
O disco de 20 GB enche	Checkpoints de ~4,2 GB, dois mais o `best.pt`: ~12,6 GB. Os dados entram como dataset e não contam para esse limite.
O PC aquece ao construir os dados	3 processos e prioridade abaixo do normal (decisão 38), com registo da temperatura, como no M7.
O PyTorch/XLA não rende nas TPUs	Fica tudo nas T4, como planeado; perdem-se só horas de quota de TPU. O caminho XLA nunca substitui o caminho CUDA.
O ganho fica abaixo do esperado	O K8 compara com o 0.1 no mesmo conjunto de validação. Mesmo um ganho pequeno deixa pronto o que o Lusitanus precisa: DDP, WSD, fp16 e o fluxo do Kaggle.

11 · DECISÕES

O que ficou decidido

A 14/11/2026 decidiste seguir as recomendações, com uma regra acima de todas: tirar o máximo de proveito do Kaggle para treinar o modelo ao máximo. Estas decisões entram na tabela de decisões do `PLAN`, como no 0.1.

D1 **Um modelo de 348M feito de raiz, ou continuar a treinar o 0.1 (125M)?**

Decidido 348M de raiz. Continuar o 0.1 fica como plano B se as T4 forem muito mais lentas do que o previsto. Se as TPUs renderem, o K5 revê o tamanho para cima.

D2 **Até onde treinar?**

Decidido Ao máximo: não parar nos 10B. Continuar até 15B, ou mais, enquanto a perda de validação descer o suficiente para compensar a quota; só então a descida final do WSD. Repetir dados até ~2 épocas é aceite.

D3 **Contexto de 2 048 ou manter 1 024?**

Decidido 2 048: +8% de computação por token, e o chat lembra-se do dobro da conversa.

D4 **Os dados constroem-se no teu PC e sobem para o Kaggle, ou num notebook de CPU do Kaggle?**

Decidido No PC, como no M7, em modo suave (3 processos, prioridade baixa): a pipeline está provada lá. Se subir ~19 GB demorar demais, passa-se para o notebook de CPU, que não gasta quota de GPU.

D5 **Português ou código já no Vulpes Flos 1.0?**

Decidido Não. Precisam de um tokenizer novo, que deita fora os 2,94B tokens já feitos e muda o chat e a avaliação. É o trabalho do Vulpes Lusitanus 1.0.

D6 **Os pesos do Vulpes Flos 1.0 são publicados?**

Em aberto, até ao K10 Como planeado para o 0.1 (GitHub Releases, Apache-2.0), ou privados. Os dados são os mesmos do 0.1, por isso não há licenças novas a pesar.

D7 **Usar também a quota de TPU do Kaggle?**

Decidido Sim, se o ensaio do K5 o justificar (página 5). O plano não depende dela: sem TPUs, tudo corre nas T4.

Próximo passo. Acabar o K1: este plano passa para `docs/` como o `PLAN.md` do Vulpes Flos 1.0. Depois, o K2 (treino v1.1) e o K3 (dados), que correm no teu PC e não custam nada. O K3 começa quando disseres, porque descarrega ~28,5 GB e ocupa o PC durante ~6 h.